

D&D 3.5e Prototype — Refactoring Guide

This document provides patterns and instructions for continuing the codebase refactoring started in Phases 1–3. It covers service extraction from `GameManager`, magic folder reorganization, and general code quality practices.

Table of Contents

1. [Service Extraction Pattern](#)
 2. [How to Extract More Services from GameManager](#)
 3. [Magic Folder Migration Pattern](#)
 4. [Code Quality Best Practices](#)
 5. [Completed Refactoring Summary](#)
-

1. Service Extraction Pattern

Overview

`GameManager` is the central coordinator for the game. Over time, it accumulated many responsibilities: combat flow, economy, AI, movement, input, conditions, spells, etc. The refactoring strategy is to **extract cohesive groups of functionality into dedicated services** while keeping `GameManager` as the lightweight coordinator.

Template: EconomyService (Phase 3)

`Assets/Scripts/Services/EconomyService.cs` was extracted as the template example. Here's the pattern:

Step 1: Create the Service Class

```
// Assets/Scripts/Services/YourNewService.cs
public class YourNewService : MonoBehaviour
{
    private GameManager _gameManager;

    /// <summary>
    /// Called by GameManager after Awake to inject dependencies.
    /// </summary>
    public void Initialize(GameManager gameManager)
    {
        _gameManager = gameManager;
    }

    public void Cleanup()
    {
        // Reset state on combat end or destruction.
    }

    // ... extracted methods go here
}
```

Step 2: Add Field + Initialize in GameManager

```
// In GameManager.cs – field declarations section (~line 220)
[SerializeField] private YourNewService _yourNewService;

// In InitializeServices() method (~line 435)
_yourNewService ??= gameObject.GetComponent<YourNewService>()
    ?? gameObject.AddComponent<YourNewService>();
_yourNewService.Initialize(this);
```

Step 3: Delegate from GameManager

Keep the **public API on GameManager** for backward compatibility, but delegate internally:

```
// Before (in GameManager):
public void DoSomething(int param)
{
    // 50 lines of logic...
}

// After (in GameManager):
public void DoSomething(int param)
{
    if (_yourNewService != null)
        _yourNewService.DoSomething(param);
    else
        DoSomethingFallback(param); // Optional fallback
}
```

Step 4: Gradually Remove Fallbacks

Once you're confident the service is stable, remove the fallback paths and make the delegation the only path.

Existing Services (for Reference)

Service	Responsibility	Pattern
TurnService	Initiative order, turn progression	MonoBehaviour, events
MovementService	Grid movement, pathfinding, AoO	MonoBehaviour, Initialize(grid, provider)
InputService	Mouse/keyboard input, UI filtering	MonoBehaviour, Register-ClickHandler
ConditionService	Combat conditions tracking	MonoBehaviour, BindTurnService
AIService	NPC AI decisions, target selection	MonoBehaviour, Initialize(gameManager)
CombatFlowService	Attack pipelines, hit resolution	MonoBehaviour, Initialize(gameManager)
EconomyService	Gold, stash, store, buy/sell	MonoBehaviour, Initialize(gm, uiProvider)
DiceService	Dice rolling (d20, d6, etc.)	Static utility class
SpellResolutionService	Spell failure, SR checks	Static utility class
SavingThrowResolver	Saving throw calculations	Static utility class

2. How to Extract More Services from GameManager

Recommended Next Extractions (Priority Order)

A. SpellApplicationService

What to extract: Spell effect application, spell resolution flow, buff/debuff management.

- Methods: `ApplySpellEffect()`, `ResolveSpell()`, `HandleSpellDamage()`, etc.
- GameManager partial files: `GameManager_NewSpells.cs` and spell-related code in main file.
- ~2,000–3,000 lines of code.

B. CombatManeuverService

What to extract: Grapple, trip, disarm, sunder, bull rush, overrun logic.

- Currently split across `GrappleSystem`, `StandardManeuvers`, `SupportActions`.
- Could be unified under one service or kept as sub-services.

C. EncounterService

What to extract: Encounter generation, enemy spawning, encounter presets, victory detection.

- Methods: `GenerateEncounter()`, `SpawnEnemies()`, `CheckCombatVictory()`, etc.

D. UICoordinationService

What to extract: UI state management, button visibility, panel coordination.

- Currently mixed into GameManager and CombatUI.

Step-by-Step Checklist for Any Extraction

1. [] **Identify** all related fields, properties, and methods in GameManager
2. [] **Create** new service file in `Assets/Scripts/Services/`
3. [] **Copy** the identified code into the service (don't cut yet)
4. [] **Add** `Initialize()` with all required dependencies
5. [] **Add** the `[SerializeField]` field in GameManager
6. [] **Wire** initialization in `InitializeServices()`
7. [] **Update** GameManager methods to delegate to the service
8. [] **Test** that everything compiles and works in Unity
9. [] **Remove** dead fallback code once verified
10. [] **Commit** with descriptive message

3. Magic Folder Migration Pattern

Current State

The `Assets/Scripts/Magic/` folder contains ~80 C# files flat in the root directory. Phase 3 created the new folder structure and moved representative files as examples.

New Folder Structure

Assets/Scripts/Magic/	
 Spells/	
  Databases/	 SpellDatabase_*.cs files (23 files)
 AreaEffects/	 *AreaEffect.cs files (12 files)
 StatusEffects/	 *EffectData.cs files (18 files)
 SpellDatabase.cs	 Main coordinator (stays here)
 SpellData.cs	 Core data structure (stays here)
 SpellCaster.cs	 Core casting system (stays here)
 SpellcastingComponent.cs	 Entity component (stays here)
 AoESystem.cs	 Core AoE system (stays here)
 AreaEffectManager.cs	 Manager (stays here)
 StatusEffectManager.cs	 Manager (stays here)
 ConcentrationManager.cs	 (stays here)
 [enums and data classes]	 BonusType.cs, SpellSchool.cs, etc. (stay here)

Files Already Migrated (Examples)

Spells/Databases:

- `SpellDatabase_A.cs` → `Spells/Databases/SpellDatabase_A.cs`
- `SpellDatabase_B.cs` → `Spells/Databases/SpellDatabase_B.cs`
- `SpellDatabase_C.cs` → `Spells/Databases/SpellDatabase_C.cs`

AreaEffects:

- FireballAreaEffect.cs → AreaEffects/FireballAreaEffect.cs
- GreaseAreaEffect.cs → AreaEffects/GreaseAreaEffect.cs
- WebAreaEffect.cs → AreaEffects/WebAreaEffect.cs

StatusEffects:

- AttributeEnhancementEffectData.cs → StatusEffects/AttributeEnhancementEffectData.cs
- BlindnessDeafnessEffectData.cs → StatusEffects/BlindnessDeafnessEffectData.cs
- GlitterdustEffectData.cs → StatusEffects/GlitterdustEffectData.cs

How to Migrate Remaining Files

1. **Move the file** to its new folder:

```
bash
```

```
mv Assets/Scripts/Magic/SpellDatabase_D.cs Assets/Scripts/Magic/Spells/Databases/
```

2. **No code changes needed** — Unity C# uses namespaces/class names, not file paths, for resolution. As long as the file is somewhere under `Assets/`, Unity will find it.
3. **Delete the old `.meta` file** (if not gitignored) and let Unity regenerate it.
4. **Verify compilation** in Unity after moving a batch of files.

Remaining Files to Migrate**SpellDatabase_*.cs → Spells/Databases/ (20 remaining):**

D, E, F, G, H, I, J, K, L, M, N, O, P, R, S, T, U, V, W, Z

AreaEffect files → AreaEffects/ (9 remaining):

DarknessAreaEffect, DaylightAreaEffect, FogCloudAreaEffect, GlitterdustAreaEffect, Obscuring-MistAreaEffect, SleetStormAreaEffect, StinkingCloudAreaEffect, WallOfFireAreaEffect, WindWallAreaEffect

EffectData files → StatusEffects/ (15 remaining):

CommandUndeadEffectData, DisguiseSelfEffectData, EmanationEffectData, ExpeditiousRetreatEffectData, FalseLifeEffectData, GhoulTouchEffectData, InvisibilityEffectData, MagicCircleEffectData, MelfsAcidArrowEffectData, ProtectionFromArrowsEffectData, ProtectionFromEnergyEffectData, ResistEnergyEffectData, ScareEffectData, SeeInvisibilityEffectData, SpectralHandEffectData

4. Code Quality Best Practices**Established in Phases 1-2**

- **DiceService for all rolls:** Use `DiceService.D20()`, `DiceService.Roll()`, etc. instead of `Random.Range()` for any D&D mechanic rolls. Visual/non-mechanical randomness can still use `UnityEngine.Random`.
- **AttackCalculator for feat modifiers:** Centralize feat-based attack/damage calculations in `AttackCalculator` instead of inline code in `CharacterController`.
- **Consistent logging:** Use `[Category]` prefixes in `Debug.Log` messages (e.g., `[Economy]`, `[Combat]`, `[LootFlow]`).

- **Null safety:** Always null-check components and references before use, especially for Unity objects.
- **Event-driven communication:** Services emit events (e.g., `OnGoldChanged` , `OnTurnStarted`) for decoupled communication rather than direct method calls where possible.

Naming Conventions

Type	Convention	Example
Services	*Service.cs	EconomyService.cs
Static utilities	Descriptive name	DiceService.cs , Saving-ThrowResolver.cs
Managers	*Manager.cs	AreaEffectManager.cs
Data classes	*Data.cs or *EffectData.cs	SpellData.cs , Glitter-dustEffectData.cs
UI classes	*UI.cs	StoreUI.cs , LootCollectionUI.cs
Systems	*System.cs	GrappleSystem.cs , AoESystem.cs

GameManager Coordination Pattern

```
GameManager (coordinator)
├─ delegates to → EconomyService (gold, stash, store)
├─ delegates to → CombatFlowService (attacks, damage)
├─ delegates to → AIService (NPC decisions)
├─ delegates to → MovementService (pathfinding, grid)
├─ delegates to → TurnService (initiative, turns)
├─ delegates to → ConditionService (buffs/debuffs)
├─ delegates to → InputService (input routing)
└─ coordinates all of the above
```

5. Completed Refactoring Summary

Phase 1: Core Service Extraction

- Extracted `DiceService` (static) for centralized dice rolling
- Extracted `AttackCalculator` for feat-based combat modifiers
- Replaced `Random.Range` calls with `DiceService` throughout

Phase 2: Combat System Cleanup

- Extracted `SavingThrowResolver` (static) for saving throws
- Extracted `SpellResolutionService` (static) for spell checks
- Extracted `CombatFlowService` for attack pipelines

- Extracted `MovementService` for grid/pathfinding logic
- Extracted `InputService` for input handling
- Extracted `ConditionService` for condition tracking
- Extracted `TurnService` for initiative/turn management
- Extracted `AIService` for NPC AI logic

Phase 3: Template Implementation (Current)

- Extracted `EconomyService` as template for future extractions
- Created Magic folder structure with example migrations
- Created this documentation (`REFACTORING_GUIDE.md`)

Future Work

- Complete Magic folder migration (44 remaining files)
- Extract `SpellApplicationService` from `GameManager`
- Extract `EncounterService` from `GameManager`
- Add unit tests for extracted services
- Remove legacy fallback code from `GameManager` once services are stable